

HACKER ATTACK

ATAQUES DE INJEÇÃO



8

LISTA DE FIGURAS

Figura 1 – Ciclo de vida de uma requisição	7
Figura 2 – Exemplo de verbo GET	9
Figura 3 – Execução do <i>exploit</i> usando o <i>eval()</i>	13
Figura 4 – Execução do <i>exploit</i> usando o <i>eval()</i>	13
Figura 5 – Execução do comando <i>dir</i> utilizando o <i>eval()</i>	14
Figura 6 – Instalação do CherryPy	15
Figura 7 – Inicialização do CherryPy	16
Figura 8 – Mensagem de <i>WebService</i> operacional.....	16
Figura 9 – Chamada do método <i>calcEnergy</i> via <i>WebService</i>	17
Figura 10 – Resultado do código injetado via <i>WebService</i>	18
Figura 11 – Resultado do código injetado via <i>WebService</i> com <i>regex</i>	21

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Exemplo de HTTP Request	8
Código-fonte 2 – Exemplo de HTTP Response.....	8
Código-fonte 3 – Sintaxe do método eval()	11
Código-fonte 4 – Primeiro exemplo de <i>exploit</i> usando o eval()	12
Código-fonte 5 – Segundo exemplo de <i>exploit</i> usando o eval()	12
Código-fonte 6 – Terceiro exemplo de <i>exploit</i> usando o eval()	13
Código-fonte 7 – Quarto exemplo de <i>exploit</i> usando o eval()	13
Código-fonte 8 – Comando para instalação do CherryPy	15
Código-fonte 9 – Código do <i>Smart Meter</i> integrado à Web.....	16
Código-fonte 10 – Chamada GET válida para o <i>WebService</i>	17
Código-fonte 11 – Chamada GET com <i>Code Injection</i> para o <i>WebService</i>	17
Código-fonte 12 – Exemplo de <i>Regular Expression</i> no Python.....	19
Código-fonte 13 – Código do <i>WebService</i> com <i>regex</i>	20

SUMÁRIO

ATAQUES DE INJEÇÃO.....	5
1 INTRODUÇÃO	5
2 SERVIÇOS WEB.....	6
3 PACOTE HTTP	7
4 WEBSERVERS	10
5 CONCEITO DE CODE INJECTION	11
6 LABORATÓRIO – CODE INJECTION	14
6.1 Regular Expressions (Regex).....	19
6.2 Aplicação de Regular Expressions.....	19
CONCLUSÃO.....	21
REFERÊNCIAS.....	22

ATAQUES DE INJEÇÃO

1 INTRODUÇÃO

Normalmente, linguagens compiladas possuem menor margem para que sejam utilizadas as técnicas de Programação Dinâmica, muito devido ao código-fonte ser transformado em código de máquina, o que torna a programação menos flexível para geração dinâmica de código, o que costuma ser mais simples em linguagens interpretadas, pois o interpretador realiza a tradução de código sob demanda.

Duas linguagens de programação foram utilizadas para avaliação dessas técnicas, primeiramente a linguagem Java, que possui várias limitações na parte de geração de código dinâmico, normalmente se limitando a fazê-lo durante a geração de *queries* e comandos externos à aplicação. E posteriormente Python, que é uma linguagem que permite ao programador realizar modificações na sua própria arquitetura, oferecendo diversas possibilidades de geração de código.

A facilidade em gerar código vem com um preço: que apareçam pontos de vulnerabilidade no sistema. O foco, que até então estava em analisar a arquitetura das linguagens de programação e de banco de dados, muda agora para analisar vulnerabilidades que são exploradas pelos *hackers* de maneira a manipular ou até mesmo ganhar acesso total ao sistema.

E a primeira dessas classes de vulnerabilidades se refere ao *Code Injection* (injeção de código), que nada mais é do que achar uma maneira de o invasor colocar um código dele para ser executado no sistema hospedeiro. Existem várias formas de se fazer isso, então nos atentaremos aos principais métodos para se fazer isso.

A base teórica para fazer esse tipo de ataque já foi vista, mas a abordagem agora concentra-se em pensar como um invasor e fazer as blindagens apropriadas no código.

Mas antes de trabalharmos com as técnicas de *Code Injection*, faremos um parêntese para vermos como é a estrutura de um serviço web ou API (*Application Programming Interface*), que é o principal meio para aplicativos *mobile* e grandes

portais na internet realizarem a comunicação com os usuários finais e, portanto, costuma ser um ponto interessante por meio do qual um invasor busca vulnerabilidades de sistema.

2 SERVIÇOS WEB

Para entender o funcionamento de um *WebService* (Serviço Web), é necessário compreender como é a sua arquitetura e depois sua estrutura interna, que não costuma apresentar muitas diferenças entre sistemas operacionais nem entre linguagens de programação.

O aspecto universal desse formato de comunicação e integração historicamente levou ao desenvolvimento acelerado da internet, pois possibilitou que inúmeros sistemas com características distintas se comunicassem.

Existem duas gerações de *WebServices* sendo utilizadas atualmente, o SOAP (*Simple Object Access Protocol*) e o REST (*Representational State Transfer*), que possuem formas bem distintas de operação.

- O SOAP é um formato mais antigo e foi a primeira geração dos *WebServices* modernos, utilizando o XML como protocolo de comunicação universal. É comumente visto em sistemas mais antigos e exige uma configuração mais extensa para realizar a integração. Como grande parte dos sistemas modernos não o utiliza mais, não entraremos em detalhes sobre o seu funcionamento.
- Já o padrão REST (ou RESTful) é utilizado nos sistemas modernos, e usa o JSON como protocolo de comunicação. Além disso, ele se utiliza de quatro verbos (GET, POST, PUT e DELETE) para que, além das informações enviadas, possa informar a ação desejada.

E tanto o SOAP quanto o REST são *WebServices* desenvolvidos por cima do protocolo HTTP (*Hypertext Transfer Protocol*), que é o protocolo de rede de alto nível que realiza toda a parte de transferência de dados entre dois pontos de rede.

Quando um usuário acessa um determinado website, existe um fluxo de vida para a requisição utilizada, resumidamente apresentado na Figura “Ciclo de vida de uma requisição”:

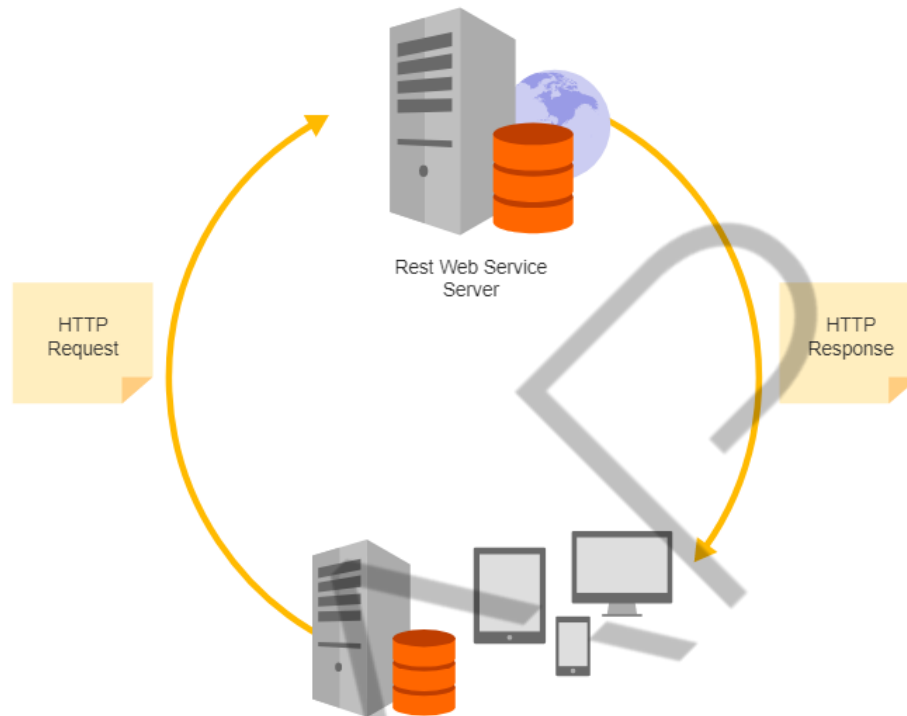


Figura 1 – Ciclo de vida de uma requisição
Fonte: Tech Tips (2018)

A requisição do usuário (*HTTP Request*) informa a URL e as informações a serem enviadas para o servidor (que pode conter um *WebService* SOAP ou RESTful), o servidor processa a requisição e monta uma resposta (*HTTP Response*), contendo as informações solicitadas ou apenas informando se a requisição foi processada com sucesso ou não.

Aparentemente, o ciclo de vida de uma requisição HTTP é bem simples olhando em alto nível, mas vale observarmos o que está por trás de um pacote HTTP, que será o alvo da próxima seção.

3 PACOTE HTTP

O pacote HTTP é a base para a realização de requisições e respostas em *WebServices*, contendo diversos campos que passam as informações necessárias

para o acesso ao *WebService* correto, assim como metadados com os dados do computador ou dispositivo que está acessando.

A seguir, é apresentado um exemplo de *HTTP Request*:

```
POST /request HTTP/1.1
Accept: application/jsonrequest
Content-Length: 72
Content-Type: application/jsonrequest
Host: json.fiap.com.br
{"user":"rafael@fiap.com.br","temperatura":27,"cep":1538001}
```

Código-fonte 1 – Exemplo de HTTP Request
Fonte: Elaborado pelo autor (2018)

Dentre os campos utilizados, os mais importantes são:

- POST: verbo (ação) utilizado no modelo REST.
- /request: caminho da URL de chamada do *WebService*.
- HTTP/1.1: protocolo e versão do protocolo.
- Accept: tipos de conteúdo que são aceitos.
- Content-Length: tamanho do pacote (em *bytes*).
- Content-Type: tipo de conteúdo que está dentro do pacote.
- Host: o servidor a ser acessado.
- Payload: conteúdo do pacote, com as informações a serem enviadas, nesse caso, em formato JSON.

O servidor irá processar a requisição e cria uma *HTTP Response*:

```
HTTP/1.1 200 OK
Content-Type: application/jsonrequest
Content-Length: 15
{"status":true}
```

Código-fonte 2 – Exemplo de HTTP Response
Fonte: Elaborado pelo autor (2018)

Entre os campos utilizados, os mais importantes são:

- HTTP/1.1: protocolo e versão do protocolo.
- 200 OK: código de resposta, existem vários códigos possíveis.

- Content-Type: tipo de conteúdo que está dentro do pacote.
- Content-Length: tamanho do pacote (em *bytes*).
- *Payload*: conteúdo do pacote, com as informações recebidas do servidor, nesse caso, em formato JSON.

Entre os códigos de retorno, existe uma tabela de códigos listando diversos *status*, alguns mais utilizados são:

- 400 Bad Request.
- 401 Unauthorized.
- 403 Forbidden.
- 404 Not Found.

A classe de códigos 4xx é de erros em relação à maneira com que o *WebService* é acessado. Existem outras classes e códigos que podem ser facilmente encontrados na WEB.

Uma observação final sobre o ciclo de vida de um *HTTP Request / HTTP Response* está no uso do GET e do POST, que são os dois verbos mais utilizados para acesso.

O browser do usuário, ao enviar informações de um formulário (por exemplo), quando utiliza o formato GET, as informações irão junto com a URL, conforme a Figura “Exemplo de verbo GET” a seguir:

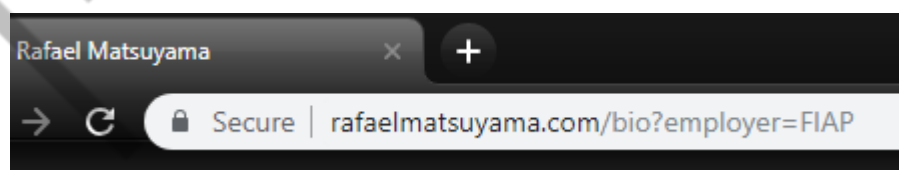


Figura 2 – Exemplo de verbo GET
Fonte: Elaborado pelo autor (2018)

Repare que, na URL, após o **?**, aparece a variável de nome **employer** e com valor **FIAP**. Após o **?**, segue uma lista de nomes de parâmetros e seus respectivos valores.

O uso desse formato de transmissão de conteúdo via URL é considerado inseguro, pois qualquer pessoa mal-intencionada que estiver observando a tela do usuário conseguirá ver rapidamente o conteúdo dos campos.

No formato POST, o conteúdo do formulário é “escondido” dentro do pacote, não se refletindo na URL, como no Código-fonte “Exemplo de HTTP Request”, por exemplo. Claro que um invasor que possua uma ferramenta como o WireShark ou outro inspecionador de pacotes de rede conseguirá ver claramente o conteúdo do formulário, mas o fato de o conteúdo não ser exibido junto com a URL no browser já dificulta esse trabalho.

4 WEBSERVERS

Nas últimas seções, foram apresentadas as questões relativas à comunicação e ao protocolo de rede, mas como funciona a etapa de processamento da requisição?

Essa etapa é responsabilidade do *WebServer*, que é uma ferramenta que gerencia o código da aplicação e controla os acessos externos e aos recursos internos do sistema. Seria o equivalente ao zelador de um prédio, em que o prédio é o sistema desenvolvido e o zelador é o responsável pelo controle de acesso de pedestres e de carros.

É extremamente importante saber a estrutura do *WebServer* utilizado no sistema a ser analisado, pois, dependendo das configurações utilizadas, podem surgir vulnerabilidades na parte de redes e de infraestrutura, que estão além do escopo de atuação do *Code Review*, mas é um assunto importante de se atentar.

No mundo Java, existem vários tipos de *WebServers*, desde os gratuitos até os pagos na sua versão corporativa. Dentre os mais utilizados, estão o Apache Tomcat, Glassfish, Jboss, WebLogic, entre outros.

Já no mundo Python, existem os *WebServers* como ferramentas separadas do código, assim como *frameworks web* que possuem um *WebServer* integrado, sendo necessário apenas utilizar alguma ferramenta de *gateway* ou *proxy* reverso como o Apache ou o Nginx, redirecionando os pedidos para o *framework web*.

No exemplo prático no final do capítulo, utilizaremos uma biblioteca bem enxuta, chamada CherryPy, que em poucas linhas de código consegue subir um *WebService* completo, mas existem outros *frameworks*, como o Django e o Flask, por exemplo.

Agora que foram apresentados os principais conceitos de infraestrutura e arquitetura de sistemas web, podemos focar nas interfaces de comunicação, por onde comumente se realiza o *Code Injection*

5 CONCEITO DE CODE INJECTION

O *Code Injection* é uma técnica que realiza, como o próprio nome diz, uma injeção de código em um sistema. Essa injeção de código não é esperada como entrada, ocorrendo muitas vezes por conta de falta de validação de campos durante a etapa de desenvolvimento ou do uso inapropriado de métodos que permitem programação dinâmica no sistema.

Existem outras classes de vulnerabilidades de injeção, como o *SQL Injection*, por exemplo. As técnicas de *SQL Injection* serão trabalhadas posteriormente, pois envolvem sintaxe específica em SQL para que sejam realizadas com sucesso.

O método **eval()** foi apresentado como uma maneira de executar código passando-se comandos no modo texto como parâmetro, e cujo uso poderia dar flexibilidade para o desenvolvedor executar rotinas dinamicamente, assim como ser um ponto de vulnerabilidade do sistema.

Vamos relembrar a sintaxe do **eval()**, pois será por meio dele que exploraremos diversas formas de fazer *Code Injection*.

```
eval(expression, globals=None, locals=None)
```

Código-fonte 3 – Sintaxe do método eval()
Fonte: Elaborado pelo autor (2018)

O **eval()** recebe uma expressão que será um texto contendo o código a ser executado, o **globals**, um dicionário de variáveis ou métodos globais permitidos e o **locals**, o dicionário respectivo, só que para escopo local.

É possível não só enviar um comando em modo texto para o **eval()** executar, como também enviar blocos de comandos e, inclusive, realizar a importação de bibliotecas nele.

Vamos trabalhar com uma série de *exploits* possíveis que podem ser feitos por cima de implementações errôneas do **eval()** e na parte prática trabalharemos blindagens mais sofisticadas a fim de evitar esses tipos de ataque.

No início, normalmente se tenta passar um código no **eval()** que realize a importação de uma biblioteca e faça alguma chamada de método, pois isso possibilita subir um terminal remoto ou habilitar ferramentas do sistema operacional.

Um exemplo de *exploit* embrionário disso está no código-fonte a seguir:

```
eval(compile('for x in range(1):\n import time\n time.sleep(5)\n print "Oi! "', 'xpto', 'single'))
```

Código-fonte 4 – Primeiro exemplo de *exploit* usando o **eval()**
Fonte: Elaborado pelo autor (2018)

Repare que foi necessário utilizar o método **compile()** para executar mais do que uma linha de código. Esse *exploit* basicamente se utiliza de um **for** para esconder dentro dele um comando **import** (que possibilitaria carregar dinamicamente alguma biblioteca) e uma chamada do método **sleep()**, ou seja, é possível acessar o método de uma biblioteca carregada com base no código injetado. Por fim, ainda é impresso o texto “Oi!” no prompt, demonstrando que é factível injetar não apenas um comando, mas uma sucessão deles.

Melhorias são possíveis com base nesse código, uma delas é injetar comandos de *shell* para executar outras aplicações, por exemplo. Um *exploit* desse tipo teria um código-fonte similar a este:

```
eval(compile("""for x in range(1):\n import os\n os.popen(r'cd').read() """, '', 'single'))
```

Código-fonte 5 – Segundo exemplo de *exploit* usando o **eval()**
Fonte: Elaborado pelo autor (2018)

Note que, ao invés de importarmos a biblioteca **time**, foi a biblioteca **os** que foi carregada. A biblioteca **os** possui métodos que executam comandos e procedimentos diretamente no sistema operacional e o método **popen()** é um desses métodos, basta informar no parâmetro o comando a ser executado na *shell* para execução.

Ao executar esse código, imprime-se na tela o diretório atual da aplicação (comando **cd**, no caso do Windows; se o sistema for Linux, use o comando **pwd** no lugar), conforme a Figura “Execução do *exploit* usando o `eval()`” a seguir:

```
C:\Python27\python.exe C:/Users/rafae/PycharmProjects/cr/code3b.py  
'C:\\Users\\rafae\\PycharmProjects\\cr\\n'
```

Figura 3 – Execução do *exploit* usando o `eval()`
Fonte: Elaborado pelo autor (2018)

É possível melhorar ainda mais esse código, vamos retirar o loop **for** desse código injetado, fazendo uso da diretiva global `__import__`, como no código-fonte a seguir:

```
eval(compile("""__import__('os').popen(r'cd').read()""", '  
, 'single'))
```

Código-fonte 6 – Terceiro exemplo de *exploit* usando o `eval()`
Fonte: Elaborado pelo autor (2018)

A execução desse código trará o mesmo resultado que o *exploit* anterior, mas utilizando menos malabarismos para realizar a mesma operação:

```
C:\Python27\python.exe C:/Users/rafae/PycharmProjects/cr/code3c.py  
'C:\\Users\\rafae\\PycharmProjects\\cr\\n'
```

Figura 4 – Execução do *exploit* usando o `eval()`
Fonte: Elaborado pelo autor (2018)

E, nesse caso, seria possível inclusive remover o comando **compile()**, pois se trata de um código relativamente simples, resultando no código-fonte a seguir:

```
print eval("""__import__('os').popen('dir').read()""")
```

Código-fonte 7 – Quarto exemplo de *exploit* usando o `eval()`
Fonte: Elaborado pelo autor (2018)

Lembrando que nem todo código consegue ser executado sem o **compile()** em conjunto com o **eval()**.

Repare que essa última versão do *exploit* é o código que será diretamente executado no **eval()** e faz a leitura de todos os arquivos no diretório da aplicação, conforme podemos ver na Figura “Execução do comando `dir` utilizando o `eval()`” a seguir:

```
7\python.exe C:/Users/rafae/PycharmProjects/cr/code3d.py
drive C has no label.
cial Number is 98E8-B1A8

of C:\Users\rafae\PycharmProjects\cr

06:51 PM    <DIR>          .
06:51 PM    <DIR>          ..
```

Figura 5 – Execução do comando dir utilizando o eval()

Fonte: Elaborado pelo autor (2018)

O comando **dir** no Windows refere-se à listagem dos arquivos de uma determinada pasta, no caso do Linux, utilize o comando **ls**.

Imagine que sejam colocados comandos de natureza mais destrutiva, como remoção de arquivos e de partições inteiras de disco, ou a instalação e execução de ferramentas como o Metasploit ou o Empire. Essa vulnerabilidade representa um sério risco para a integridade do sistema, pois não apenas permite que um invasor tome conta de um sistema, como também se utilize de um simples browser para realizar o ataque, conforme veremos na parte prática.

6 LABORATÓRIO – CODE INJECTION

Vamos trabalhar com uma extensão do exercício prático vista anteriormente, na qual utilizamos um *Smart Meter* que reportava o valor do consumo de energia e de água de um determinado consumidor.

Recapitulando as blindagens feitas, a primeira delas é não utilizar o método **eval()** para o **condições do else**, pois se poderia tirar proveito de uma informação mal estruturada para injetar código.

Depois foram apresentadas duas possibilidades, reduzir o escopo do método **eval()** ou remover a chamada do método **eval()**.

Neste exercício prático, vamos colocar o *Smart Meter* como um medidor que está exposto à rede como um *WebService* RESTful, utilizando o CherryPy como *WebServer*. Mas, antes, vamos instalar e configurar o CherryPy.

Para realizar a instalação do CherryPy, basta realizar a instalação da biblioteca via **pip install**, utilizando o seguinte comando:

```
pip install cherrypy
```

Código-fonte 8 – Comando para instalação do CherryPy
Fonte: Elaborado pelo autor (2018)

O CherryPy precisa que outras bibliotecas sejam pré-instaladas, após a sua instalação, uma tela de sucesso semelhante à da Figura “Instalação do CherryPy” a seguir será exibida:

```
2.4->cherrypy) (1.3)
Collecting jaraco.functools>=1.20 (from tempora>=1.8->portend>=2.1.1->cherrypy)
  Downloading https://files.pythonhosted.org/packages/c2/f3/ce1bd8d5434227d26c2c2e07a9f89229a6b8112243745287e67f0ec76/jaraco.functools-1.20-py2.py3-none-any.whl
Requirement already satisfied: pytz in c:\python27\lib\site-packages (from tempora>=1.8->portend>=2.1.1->cherrypy) (2018.5)
Installing collected packages: contextlib2, more-itertools, jaraco.functools, tempora, portend, zc.lockfile, win32, cheroot, cherrypy
  Running setup.py install for zc.lockfile ... done
Successfully installed cheroot-6.5.2 cherrypy-17.4.0 contextlib2-0.5.5 jaraco.functools-1.20 more-itertools-0.0 portend-2.3 pywin32-224 tempora-1.14 zc.lockfile-1.3.0
```

Figura 6 – Instalação do CherryPy
Fonte: Elaborado pelo autor (2018)

Caso tenha problemas na instalação, a documentação oficial no site do CherryPy (<https://cherrypy.org>) é bem completa.

O código-fonte do nosso *Smart Meter* integrado à internet consta a seguir:

```
import cherrypy

class SmartMeter(object):
    @cherrypy.expose
    def index(self, metodo="", data=0, price=0.0):
        if (metodo == ""):
            return "WebService Online!"
        elif (metodo == 'calcEnergy'):
            return "A fatura de energia foi de R$ " +
str(eval(metodo + "(%s, %s)" % (data, price)))
        elif (metodo == 'calcWater'):
            return "A fatura de agua foi de R$ " +
str(eval(metodo + "(%s, %s)" % (data, price)))
        else:
            print(metodo)
            return eval(metodo)

    def calcEnergy(kwh, price):
        return kwh * price

    def calcWater(m3, price):
```

```
        return m3 * price

if __name__ == '__main__':
    cherrypy.quickstart(SmartMeter(), '/')
```

Código-fonte 9 – Código do *Smart Meter* integrado à Web

Fonte: Elaborado pelo autor (2018)

Dentro do método **index()**, é desenvolvida a mesma lógica de negócio que a do exercício prático anterior, em que a leitura do método é feita e, com base no seu valor, é chamado o **calcEnergy** ou o **calcWater** ou o próprio método no **else**. Esse método está anotado com o **@cherrypy.expose** para externalizar esse método como acessível pela web.

No **if** ao final do código, é feita a execução do CherryPy que instancia o *WebService SmartMeter* para a web.

Ao executar esse código no PyCharm, algumas mensagens de log de inicialização do CherryPy serão exibidas, numa delas constará o endereço URL em que o servidor estará no ar, mensagens parecidas com esta:

```
08:29:32] ENGINE Set handler for console events.
08:29:32] ENGINE Started monitor thread 'Autoreloader'.
08:29:32] ENGINE Serving on http://127.0.0.1:8080
08:29:32] ENGINE Bus STARTED
```

Figura 7 – Inicialização do CherryPy

Fonte: Elaborado pelo autor (2018)

Agora o *WebService* está acessível pela web, basta entrar no endereço especificado no log (<http://127.0.0.1:8080>) e veremos uma mensagem de que o serviço está on-line.

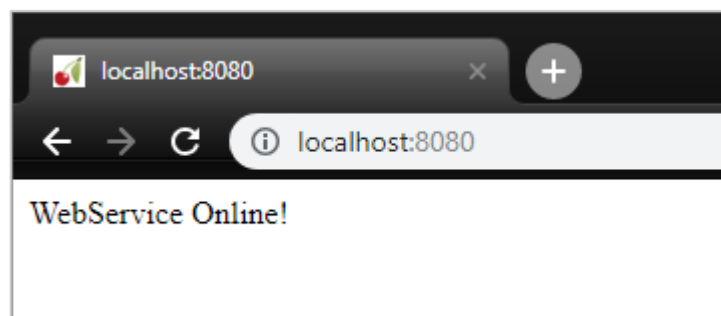


Figura 8 – Mensagem de *WebService* operacional

Fonte: Elaborado pelo autor (2018)

A partir desse endereço, podemos fazer as chamadas, passando-se os valores para o *WebService*; testaremos primeiro com valores válidos antes de fazer injeção de código nele.

```
http://localhost:8080/?metodo=calcEnergy&data=150&price=4.50
```

Código-fonte 10 – Chamada GET válida para o *WebService*
Fonte: Elaborado pelo autor (2018)

E abrindo essa URL no browser, teremos a conta de energia sendo realizada pelo método **calcEnergy()**.

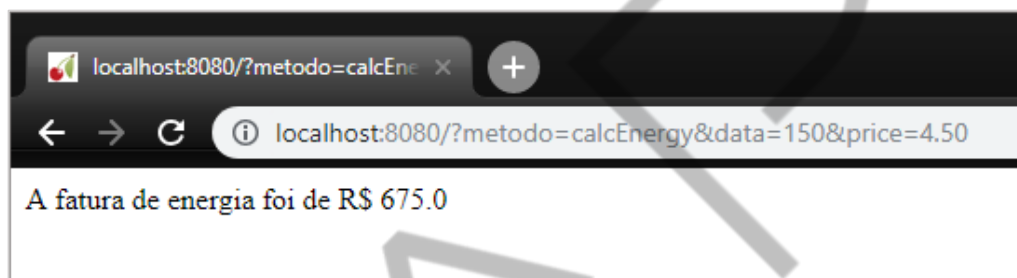


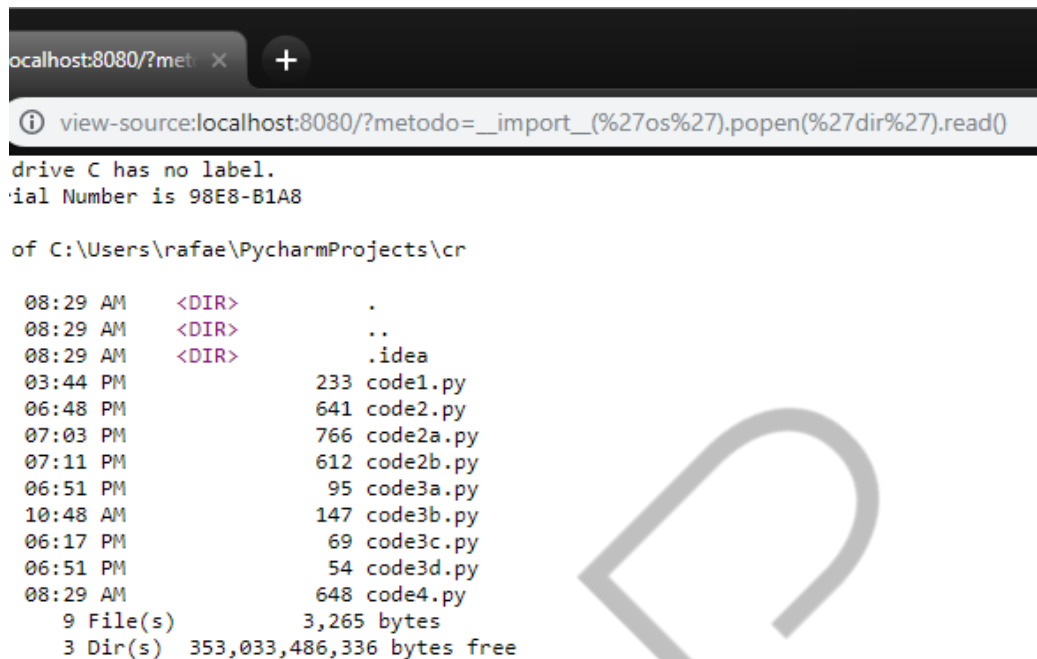
Figura 9 – Chamada do método calcEnergy via *WebService*
Fonte: Elaborado pelo autor (2018)

Agora vamos fazer uma injeção de código nesse *WebService* alterando o valor do parâmetro **metodo** para executar um comando de *shell* nele.

```
http://localhost:8080/?metodo=__import__(%27os%27).popen(%27dir%27).read()
```

Código-fonte 11 – Chamada GET com *Code Injection* para o *WebService*
Fonte: Elaborado pelo autor (2018)

Repare que, na URL, é passado o mesmo comando que utilizamos para fazer a injeção de código anteriormente e agora veremos o resultado disso no próprio browser:



```
localhost:8080/?meto=...  
view-source:localhost:8080/?metodo=__import__(%27os%27).popen(%27dir%27).read()  
drive C has no label.  
ial Number is 98E8-B1A8  
of C:\Users\rafae\PycharmProjects\cr  
  
08:29 AM <DIR> .  
08:29 AM <DIR> ..  
08:29 AM <DIR> .idea  
03:44 PM 233 code1.py  
06:48 PM 641 code2.py  
07:03 PM 766 code2a.py  
07:11 PM 612 code2b.py  
06:51 PM 95 code3a.py  
10:48 AM 147 code3b.py  
06:17 PM 69 code3c.py  
06:51 PM 54 code3d.py  
08:29 AM 648 code4.py  
9 File(s) 3,265 bytes  
3 Dir(s) 353,033,486,336 bytes free
```

Figura 10 – Resultado do código injetado via *WebService*
Fonte: Elaborado pelo autor (2018)

Observação: para o leitor mais atento, foi utilizado o modo de visualização do código-fonte para conseguir ver a listagem do diretório de maneira mais agradável, pois o resultado do comando não está em formato HTML.

Note que não é difícil fazer injeção de código em um *WebService*, a dificuldade maior está em descobrir alguns aspectos do seu código, como:

- A linguagem de programação utilizada.
- Se é usada alguma forma de programação dinâmica ou não.
- Se é feita validação dos parâmetros.

E esses aspectos deverão ser observados por quem está realizando o procedimento de *Code Review*, mas nenhuma das blindagens oferecidas até agora resolvem propriamente a questão da injeção de código, as alternativas propostas até o momento cobrem apenas o **eval()** na sua redução de escopo ou remoção.

Agora veremos outra forma de tratar esse problema, que tal fazer o código analisar e buscar determinados comandos dentro dos parâmetros e evitar que estes sejam executados?

Para isso, teremos que ver um pouco sobre o uso de *Regex (Regular Expressions)*.

6.1 Regular Expressions (Regex)

Regex é uma funcionalidade que está presente em inúmeras linguagens de programação desde os primórdios da computação, que realiza a detecção de padrões em textos conforme determinadas regras que o programador informar.

Existem manuais e livros com inúmeros exemplos de padrões que podem ser detectados com o *regex*, geralmente qualquer regra de texto que possa ser expressa como lógica booleana é passível de ser analisada por essa funcionalidade.

Normalmente, parte-se de um texto antes de se montar as regras de detecção, para isso utiliza-se ferramentas como o Regexr (<https://regexr.com/>), para saber se os filtros criados são adequados antes de desenvolver o código de detecção.

No Python, o *regex* está presente por meio da biblioteca **re**, na qual podemos compilar um padrão de texto a ser detectado, conforme o código-fonte a seguir:

```
import re

p = re.compile('[a-zA-Z]*ex[a-zA-Z]*')
print p.findall("Texto de exemplo")
```

Código-fonte 12 – Exemplo de *Regular Expression* no Python
Fonte: Elaborado pelo autor (2018)

Nesse código, existe o método **compile()**, que guarda o padrão a ser detectado (que seriam palavras contendo o texto **ex**, no exemplo) e o método **findall()**, que realiza a busca do padrão dentro do texto, que trará como resultado duas respostas: texto e exemplo, pois são as duas palavras que se encaixam no padrão informado.

6.2 Aplicação de Regular Expressions

Uma maneira interessante de realizar a blindagem contra injeção de código seria aplicar as *Regular Expressions* de maneira que, ao detectar a presença de um comando nocivo dentro dos parâmetros informados ao *WebService*, impeça o processamento desse código.

O comportamento do *regex* nesse contexto é similar ao comportamento de um *firewall*, no sentido de que o *regex* analisa e filtra as informações que entram em um sistema, da mesma forma que o *firewall* faz com as conexões e os fluxos de rede.

Colocando o *regex* no *WebService*, vamos filtrar por termos como **import**, **popen** e **os**.

```
import cherrypy, re

class SmartMeter(object):
    @cherrypy.expose
    def index(self, metodo="", data=0, price=0.0):
        p = re.compile('(import|popen|os)')
        if p.findall(metodo) is not None:
            return "Code Injection!"

        if (metodo == ""):
            return "WebService Online!"
        elif (metodo == 'calcEnergy'):
            return "A fatura de energia foi de R$ " +
str(eval(metodo + "(%s, %s)" % (data, price)))
        elif (metodo == 'calcWater'):
            return "A fatura de agua foi de R$ " +
str(eval(metodo + "(%s, %s)" % (data, price)))
        else:
            return eval(metodo)

    def calcEnergy(kwh, price):
        return kwh * price

    def calcWater(m3, price):
        return m3 * price

if __name__ == '__main__':
    cherrypy.quickstart(SmartMeter(), '/')
```

Código-fonte 13 – Código do *WebService* com *regex*
Fonte: Elaborado pelo autor (2018)

Esse código possui apenas a parte de *regex* sendo incluída no início do método **index()**, de forma que se algum dos termos nocivos for encontrado, retorna uma mensagem de erro, como podemos observar a seguir:

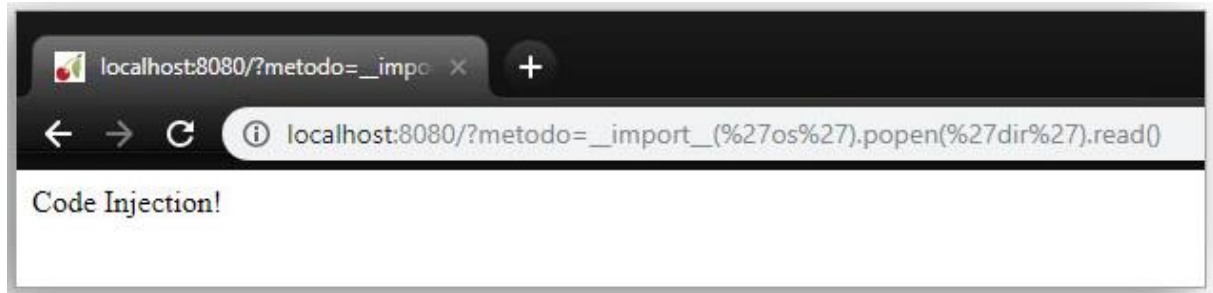


Figura 11 – Resultado do código injetado via *WebService* com *regex*
Fonte: Elaborado pelo autor (2018)

É necessário ter conhecimento da arquitetura da linguagem de programação para que essa blindagem funcione adequadamente, devido ao fato de que o *regex* precisa receber o padrão a ser detectado para que consiga evitar que ocorra injeção de código.

CONCLUSÃO

Code Injection (injeção de código) é uma vulnerabilidade muito comum em sistemas que não realizam a devida validação nos parâmetros recebidos pelo usuário ou por interfaces externas.

Existem várias maneiras de se blindar o código para evitar ou ao menos reduzir as chances de ocorrência, desde reduzir o escopo ou eliminar o componente de programação dinâmica, mas quando isso não é possível, utiliza-se *Regular Expressions* (*regex*) para detectar palavras-chave que possam ser parte de comandos nocivos ao sistema.

A blindagem é importante apenas nas interfaces para a web? Não, a sua importância abrange as integrações internas da sua aplicação, como, por exemplo, nas consultas ao banco de dados.

REFERÊNCIAS

TECH TIPS. **GetResponseStream Reading Response From URL using HTTP Web Request.** Disponível em: <<https://learn-tech-tips.blogspot.com/2016/04/getresponsestream-post-get-method-csharp.html>>. Acesso em: 7 nov. 2018.

EMASP